

# Fixed Geometric Routing from Angular Structure

Casey Allard  
Independent Researcher

March 2026

## Abstract

Sparse expert routing is typically learned by a gating network. This paper studies a narrower alternative: whether a fixed routing map derived from embedding geometry can provide a useful routing scaffold without a learned gate matrix. We evaluate a Hopf-derived routing map built from a fixed 4D projection of normalized representations. On a semantic proxy based on structured embeddings, the method yields a sublinear effective routing footprint with canonical fit  $\hat{e} = 2.957K^{0.572}$  over  $K = 25\text{--}400$ , versus linear scaling for dense routing, and the advantage persists through  $K = 5000$ . The proxy result is strengthened by explicit controls: concentration weakens under column permutation of the embeddings, and fixed Hopf routing is more concentrated than adaptive  $K$ -means on the same proxy. In a toy two-layer language model, fixed Hopf routing partially transfers: on both Penn Treebank and WikiText-2 it stays within about 8% of a learned top-1 baseline while removing the learned gate matrix. The key limitation is also the key null result: in end-to-end LM training, Hopf routing is not meaningfully better than randomly permuted fixed routing. The resulting conclusion is intentionally narrow. Angular structure is sufficient to support concentrated fixed routing on a semantic proxy, and fixed routing can partially transfer to toy LM training, but the present evidence does not establish a geometry-specific quality advantage in trainable language models.

## 1 Introduction

Sparse routing is attractive because it reduces active computation, but standard expert models pay for that sparsity with a learned gate, balancing heuristics, and their own optimization pathologies (Shazeer et al., 2017; Fedus et al., 2022). Paper 1 asks a narrower question than whether geometry can replace modern mixture-of-experts systems wholesale. The question here is whether some routing structure can be supplied directly by the geometry of normalized representations.

The paper focuses on a fixed Hopf-derived routing map. Each input is normalized, projected onto a fixed 4D subspace, converted to low-dimensional angular coordinates, and discretized into a routing sector. No gate matrix is learned. This idea is motivated by prior evidence that normalized language representations are angularly structured rather than uniform on the sphere (Google Research, 2026). If the data distribution is anisotropic, a fixed sector map may receive uneven traffic,

concentrating computation into a subset of sectors.

The contribution is deliberately scoped to three points.

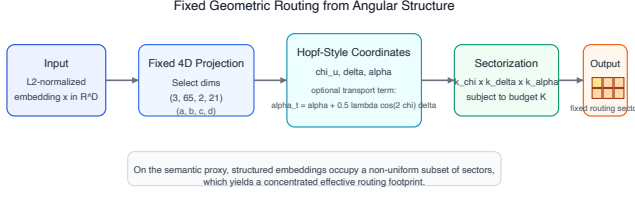
- On a semantic proxy built from structured embeddings, fixed Hopf routing yields a sublinear effective routing footprint and remains sublinear through large routing budgets.
- In a toy two-layer language model, the same fixed routing mechanism partially transfers, staying within roughly 8% of a learned top-1 baseline on Penn Treebank and WikiText-2 while removing the learned gate matrix.
- The trainable LM result is bounded by a null finding: Hopf routing is not distinguishable from randomly permuted fixed routing on quality in end-to-end training.

The strongest evidence is proxy-side rather than LM-side. The paper therefore emphasizes mechanism, controls, and honest claim boundaries over broad generalization.

## 2 Problem Setup and Routing Objective

The object under test is a *fixed sector map* on normalized representations, not a general manifold-native transformer. The central quantity is the *effective routing footprint*: the effective number of routing sectors or expert paths used under the empirical routing distribution. In the canonical analyses this appears as `effective_bucket_count` on the proxy and `eff_buckets` in the language-model setting. At fixed routing budget  $K$ , lower values indicate more concentrated routing.

We study this quantity in two settings. The first is a semantic proxy built from static structured embeddings. This setting isolates routing geometry from end-to-end expert adaptation and is therefore the main testbed for whether the routing map itself concentrates traffic. The second is a toy trainable language model, where fixed routing is compared against a learned top-1 baseline and a randomly permuted fixed-routing null. These two settings answer different questions: the proxy asks whether angular geometry induces concentration, while the LM asks whether a fixed routing scaffold remains usable when experts are trained end to end.



**Figure 1:** Fixed routing mechanism used in Paper 1. A normalized representation is projected onto a fixed 4D subspace, expressed in Hopf-style angular coordinates, optionally transport-corrected, and discretized into a sector under routing budget  $K$ . The positive proxy result is that structured embeddings occupy these sectors non-uniformly.

## 3 Fixed Geometric Routing Method

### 3.1 Why a Fixed 4D Projection?

The routing function starts from a normalized input  $\mathbf{x} \in \mathbb{R}^D$  and applies a fixed coordinate projection to four selected dimensions. In the canonical configuration this tuple is (3, 65, 2, 21), giving projected coordinates  $(a, b, c, d)$ . The choice is pragmatic rather than metaphysical. A four-coordinate map is small enough that its geometry can be written down explicitly, plotted, and discretized into routing sectors, while still leaving enough degrees of freedom for nontrivial angular structure.

This matters because Paper 1 is not testing whether routing can be learned from geometry in the abstract. It is testing whether a fixed, inspectable geometric address can act as a routing scaffold at all. The 4D projection is the simplest configuration in the evidence base that supports the main empirical story: it yields the canonical proxy scaling law, weakens under structure-destroying controls, and transfers as a fixed scaffold into the toy language model.

### 3.2 Hopf Coordinates and Their Interpretation

Figure 1 shows the mechanism. After L2 normalization and fixed 4D projection, the method computes three angular quantities:

$$\rho_1 = \sqrt{a^2 + b^2}, \quad \rho_2 = \sqrt{c^2 + d^2}, \quad (1)$$

$$\chi_u = \frac{\rho_2^2}{\rho_1^2 + \rho_2^2}, \quad (2)$$

$$\delta = \arctan 2(b, a) - \arctan 2(d, c), \quad (3)$$

$$\alpha = \frac{1}{2}(\arctan 2(b, a) + \arctan 2(d, c)). \quad (4)$$

These coordinates give a routing address with a readable interpretation. The coordinate  $\chi_u$  is a coarse base variable that records how energy is split between the two projected planes. The difference term  $\delta$  records how those planes are oriented relative to one another. The phase term  $\alpha$  acts like a mean angular address. In other words, the routing mechanism is not learning a cluster assignment from scratch; it is decomposing the projected representation into a base coordinate, a relative-angle coordinate, and a phase coordinate that can be discretized directly.

### 3.3 What the Transport Term Changes

The canonical routing rule includes a transport correction on the phase coordinate:

$$\chi = \arcsin \left( \frac{\rho_2}{\sqrt{\rho_1^2 + \rho_2^2}} \right), \quad \alpha_t = \alpha + \frac{1}{2} \lambda \cos(2\chi) \delta. \quad (5)$$

This term does not carry an independent headline claim, but it matters for what address the routing rule actually uses. Without the correction, the phase coordinate is only the average of the two planar angles. With the correction, the phase is perturbed by the local geometry of the point through  $\chi$  and  $\delta$ . Two inputs with similar raw phase can therefore be separated if they sit in different parts of the projected geometry. The right interpretation for Paper 1 is modest: the transport term sharpens the phase address used for sectorization, rather than defining a separate learning objective or a separate notion of routing quality.

### 3.4 Why Sectorization Can Concentrate Routing

Given routing budget  $K$ , the coordinate triple  $(\chi_u, \delta, \alpha_t)$  is discretized into product bins  $(k_\chi, k_\delta, k_\alpha)$  with total budget at most  $K$ . The resulting bin index is the routing address. Because this address is fixed, any concentration must come from the data distribution rather than from learned gating.

That distinction is central to the paper. If normalized embeddings were close to uniform in these coordinates, then increasing  $K$  would simply expose more sectors and the effective routing footprint would grow almost linearly with budget. The proxy results show the opposite. Structured embeddings accumulate in a subset of sectors, the growth rate is sublinear, and the effect weakens under controls that destroy or weaken the underlying structure. The routing footprint is therefore not being manufactured by a trainable gate; it is a measurable consequence of how the embedding distribution sits inside the fixed coordinate system.

## 4 Experimental Setup

The paper uses two complementary settings. The semantic proxy is the primary geometry testbed. It uses a WikiText-2-derived PPMI-SVD embedding proxy, which preserves structured static embedding geometry while removing end-to-end expert adaptation. That makes it the right place to test whether the routing map itself produces concentrated traffic. The toy language model is a transfer setting only. A compact setup table is provided in Appendix A.

### 4.1 Semantic Proxy

The semantic proxy analyses draw from canonical results on a WikiText-2-derived PPMI-SVD embedding dataset. The routing budget ranges from  $K = 25$  through  $K = 5000$ . The proxy does not involve LM backpropagation, which is a feature

rather than a defect for this paper’s first-stage claim: it isolates whether angular geometry alone yields concentrated routing. The main proxy metrics are effective routing footprint, sector entropy, and sector mass concentration as summarized by sector  $p_{\max}$ .

## 4.2 Toy Language Model

The LM setting uses a 2-layer transformer-like model with hidden dimension 128, 4 attention heads, vocabulary size 5000, sequence length 32, 64 experts, and expert dimension 128. All LM conditions train for 4000 steps with two seeds per dataset. Routing budget is fixed at  $K = 64$  experts. The paper compares three conditions:

- **BASELINE**: learned top-1 routing without auxiliary load-balancing loss;
- **HOPF**: fixed Hopf-derived routing with no learned gate matrix;
- **PERMUTED**: a random permutation of the same fixed scaffold.

The learned baseline omits an auxiliary balance loss intentionally. The paper’s target comparison is against a naturally concentrated top-1 regime, not against a forced-uniform sparse regime. Section 6.3 makes that design choice explicit with a dedicated control.

## 5 Semantic Proxy Results

### 5.1 Main Scaling Law

Figure 2 is the main result in the paper. On the semantic proxy, dense routing scales linearly with budget by definition. Fixed Hopf routing scales much more slowly: over the canonical anchor range  $K = 25$ –400, the fit is

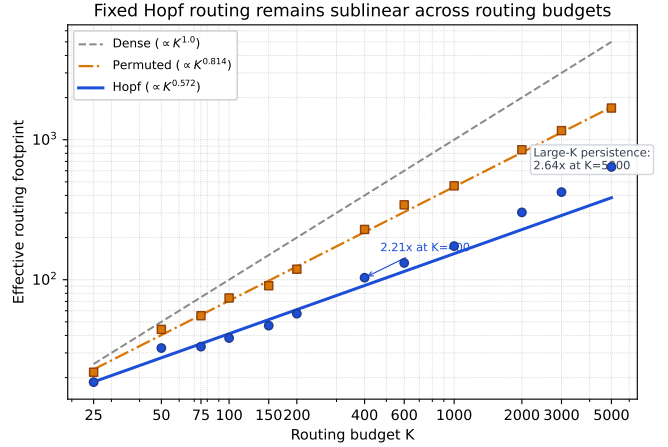
$$\hat{e} = 2.957K^{0.572},$$

while the permuted fixed-routing control scales with exponent 0.814. The core interpretation is that the routing map is aligned with structure in the embeddings rather than merely producing sparse-looking histograms at one budget.

Two details matter. First, the advantage does not collapse at larger budgets: the large- $K$  extension remains sublinear through  $K = 5000$ , with refit exponent approximately 0.657. Second, the PERMUTED control preserves the same budget and fixed-routing scaffold while destroying geometry-specific assignment. Its steeper scaling therefore supports the claim that concentration depends on structured angular signal rather than on fixed sparsity in the abstract.

### 5.2 Structural Controls and Falsifications

The main scaling law is strengthened by explicit controls, but those details fit better in the appendix than in the body. Two controls matter most. First, concentration weakens when the



**Figure 2:** Primary proxy result. Fixed Hopf routing yields a sublinear effective routing footprint on the semantic proxy. Points and fits are generated directly from canonical analysis JSON under `paper1_assets/`. The permuted control scales more steeply, and the advantage persists through large budgets, reaching  $2.21\times$  at  $K = 400$  and  $2.64\times$  at  $K = 5000$ .

structured proxy embeddings are column-permuted before routing: sector  $p_{\max}$  falls from 0.090 to 0.061, with a corresponding increase in entropy. This is the simplest structure-destroying check in the paper. Second, fixed Hopf routing is more concentrated than adaptive  $K$ -means on the same proxy, with sector  $p_{\max}$  0.087 versus 0.072. Taken together, these controls constrain the interpretation of Figure 2: the effect depends on structured angular signal and is not reducible either to arbitrary sparsity or to a generic Euclidean clustering baseline. Exact summaries are moved to Appendix A.

### 5.3 Norm Robustness

The canonical scaling exponent for Hopf routing is also stable across L1, L2, L3, and L4 normalizations. Across these four settings, the Hopf exponent varies by less than 0.000. This does not prove a deeper invariance principle, but it does support the narrower claim the paper actually needs: the main proxy result is not an artifact of one specific normalization choice and is more naturally explained as an angular effect than as a radial-shell effect. The full summary appears in Appendix A.

## 6 Trainable Language-Model Results

### 6.1 Transfer to PTB and WT2

The language-model experiments ask a narrower question than the proxy study. The proxy isolates whether geometry alone can induce concentrated routing. The trainable setting asks whether the same fixed routing scaffold remains usable once expert weights are trained end to end. The goal is not to show that fixed routing outperforms learned gating on quality, but to determine whether a fixed geometric scaffold can support a working sparse-routing regime without a learned gate matrix.

Table 1 summarizes the main results. On Penn Treebank, fixed Hopf routing reaches a HOPF/BASELINE validation-

**Table 1:** Main toy language-model results. Fixed Hopf routing stays within about 8% of the learned top-1 baseline on both datasets while removing the learned gate matrix. The same table also shows the key null result: quality is not improved relative to randomly permuted fixed routing.

| Dataset | Condition | Val. PPL | Eff. paths | 64/eff <sub>b</sub> |
|---------|-----------|----------|------------|---------------------|
| PTB     | BASELINE  | 152.26   | 43.19      | 1.48×               |
| PTB     | HOPF      | 164.54   | 45.96      | 1.39×               |
| PTB     | PERMUTED  | 164.41   | 45.81      | 1.40×               |
| WT2     | BASELINE  | 105.85   | 35.37      | 1.81×               |
| WT2     | HOPF      | 114.45   | 40.94      | 1.56×               |
| WT2     | PERMUTED  | 114.48   | 42.28      | 1.51×               |

perplexity ratio of 1.081. On WikiText-2, the corresponding ratio is 1.081. The close agreement across datasets matters more than the absolute perplexity values alone: the same fixed scaffold incurs nearly the same relative cost under the same training recipe on both corpora, which makes the transfer result more credible than a single-dataset comparison.

The routing-efficiency side is modest but consistent. Hopf uses 46.0 effective expert paths on PTB and 40.9 on WT2 out of 64. This is much weaker evidence than the proxy scaling law, and the paper should be read that way. Still, it shows that a fixed routing scaffold can remain operational in a trainable language model without catastrophic degradation or collapse to unusable routing behavior.

## 6.2 Null Result Against Permuted Fixed Routing

The central limitation of the trainable result is also its most important negative finding. HOPF and PERMUTED are effectively indistinguishable on validation perplexity in both datasets: the difference is +0.13 on PTB and -0.03 on WT2. The compact null-result table is moved to Appendix A so the main paper can keep a cleaner flow, but the interpretation belongs in the body.

This null result sharply narrows what the paper can claim. In the semantic proxy, the structured embedding distribution interacts meaningfully with the Hopf-derived sector map. In the trainable language model, that geometry-specific advantage does not survive optimization. The most plausible interpretation is that once the experts are trainable, they co-adapt to the fixed routing scaffold they are given. Under that co-adaptation, the distinction between a geometrically motivated fixed map and a randomly permuted fixed map becomes negligible on quality. The evidence therefore supports a claim about fixed routing as a usable scaffold, not a claim that Hopf geometry is uniquely superior as a trained routing rule.

## 6.3 Baseline Rationale and Auxiliary-Loss Control

The learned baseline should also be interpreted carefully. Paper 1 uses a learned top-1 gate *without* auxiliary load-balancing loss. That choice is deliberate: the paper asks whether fixed routing can replace a naturally concentrated top-1 routing regime, not whether it can replace a deliberately balanced one.

In the canonical auxiliary-loss control, adding the balance objective drives learned sparse routing toward near-uniform usage, with 62.8 effective paths out of 64. That regime is less concentrated than the no-aux baseline and substantially less aligned with the question studied here. The auxiliary-loss result is therefore part of the trust-building evidence base, but it serves mainly to justify comparator choice rather than to extend the headline claim.

## 7 Discussion and Limitations

The results support a constrained interpretation. First, fixed Hopf-derived routing yields a clearly sublinear routing footprint on a semantic proxy, and that concentration weakens under controls that destroy or ignore the underlying structure. Second, the same fixed-routing mechanism partially transfers to a toy trainable language model, remaining within roughly 8% of a learned top-1 baseline on both PTB and WT2 while removing the learned gate matrix. Third, that trainable result is bounded by a null finding: Hopf routing is not better than randomly permuted fixed routing after end-to-end expert adaptation.

Taken together, those results suggest that angular structure can provide a useful fixed routing scaffold, but they do not support a stronger claim about geometry-specific quality gains in trained language models. The proxy and trainable settings are therefore doing different scientific jobs in the paper: the proxy isolates the geometric concentration effect itself, while the language model tests whether a fixed scaffold remains usable once optimization and expert co-adaptation are introduced.

Those boundaries imply several limitations.

- The strongest evidence is proxy-side. The language-model result is a transfer test on a toy two-layer architecture, not a modern large-scale sparse transformer.
- The LM experiments test FFN routing only. They do not address attention routing or broader inference-time compute allocation.
- The paper establishes concentrated routing and partial transfer, but not end-to-end hardware savings in a trained production system.
- The geometry-specific advantage disappears under expert co-adaptation. This is a substantive negative result, not an implementation caveat.

These limitations are not side conditions to be hidden. They are part of the paper’s contribution, because they define exactly what the present evidence does and does not justify.

## 8 Conclusion

This paper presents a narrow empirical result about fixed routing from embedding geometry. A Hopf-derived routing map built from normalized representations yields a sublinear effective routing footprint on a semantic proxy, retains that advantage through large routing budgets, and weakens under controls that destroy or ignore structure. The same scaffold partially transfers

to a toy trainable language model without a learned gate matrix. At the same time, the trainable LM experiments show that Hopf routing does not outperform randomly permuted fixed routing on quality. The appropriate conclusion is therefore limited but concrete: angular structure is sufficient to support concentrated fixed routing, but the present evidence does not establish a geometry-specific advantage in end-to-end language-model training.

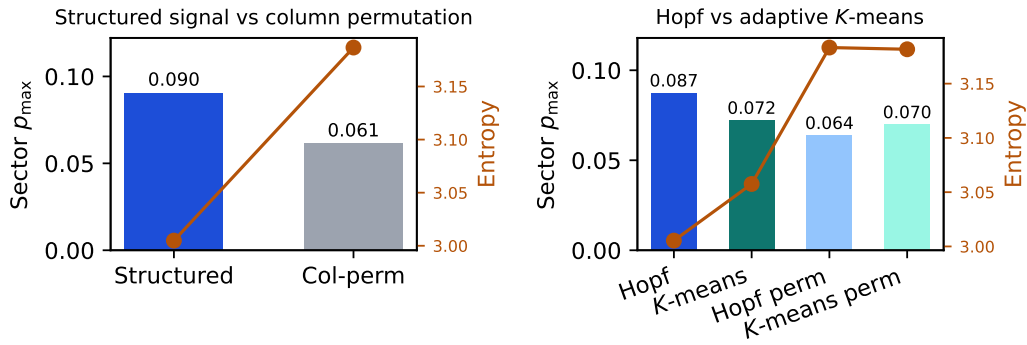
## References

- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23:1–39, 2022.
- Google Research. TurboQuant: Extreme KV-cache compression via angular rotation-invariant quantization. *arXiv preprint*, 2026.
- Mitchell P. Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. Building a large annotated corpus of English: The Penn Treebank. *Computational Linguistics*, 19(2):313–330, 1993.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint*, 2016.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.

**Table 2:** Exact LM setup values used in the canonical toy-language-model analyses.

| Hyperparameter   | Value                     |
|------------------|---------------------------|
| Layers           | 2                         |
| Hidden dimension | 128                       |
| Attention heads  | 4                         |
| Vocabulary size  | 5000                      |
| Sequence length  | 32                        |
| Experts          | 64                        |
| Expert dimension | 128                       |
| Training steps   | 4000                      |
| Datasets         | Penn Treebank, WikiText-2 |
| Conditions       | BASELINE, HOPF, PERMUTED  |

Proxy controls show concentration follows structured geometry



**Figure 3:** Proxy-side control figure moved to the appendix for layout stability. Left: concentration weakens when structured embeddings are column-permuted before routing. Right: fixed Hopf routing is more concentrated than adaptive  $K$ -means on the same proxy, and both families weaken under permutation controls.

## A Appendix: Setup and Control Summaries

**Table 3:** Structured-versus-permuted proxy control. Concentration weakens when the proxy embeddings are column-permuted before routing.

| Proxy condition | Runs | Sector $p_{\max}$ | Sector entropy | Test MSE |
|-----------------|------|-------------------|----------------|----------|
| Structured      | 4    | 0.090             | 3.005          | 0.0040   |
| Column-permuted | 4    | 0.061             | 3.187          | 0.0040   |

**Table 4:** Hopf-versus- $K$ -means proxy control. Fixed Hopf routing is more concentrated than adaptive  $K$ -means on the same structured proxy, and both weaken under permutation.

| Routing family      | Runs | Sector $p_{\max}$ | Sector entropy | Test MSE |
|---------------------|------|-------------------|----------------|----------|
| Hopf                | 2    | 0.087             | 3.005          | 0.0040   |
| $K$ -means          | 2    | 0.072             | 3.058          | 0.0040   |
| Hopf permuted       | 2    | 0.064             | 3.183          | 0.0040   |
| $K$ -means permuted | 2    | 0.070             | 3.182          | 0.0040   |

**Table 5:** Norm robustness of the proxy scaling law. The Hopf exponent remains essentially unchanged across L1, L2, L3, and L4 normalization.

| Normalization | Hopf exponent | Permuted exponent | Base exponent |
|---------------|---------------|-------------------|---------------|
| L1            | 0.572         | 0.809             | 0.916         |
| L2            | 0.572         | 0.814             | 0.916         |
| L3            | 0.572         | 0.815             | 0.916         |
| L4            | 0.572         | 0.816             | 0.916         |

**Table 6:** Compact null-result summary for the trainable LM.

| Dataset | HOPF / BASE | HOPF - PERM | HOPF eff. paths | PERM eff. paths |
|---------|-------------|-------------|-----------------|-----------------|
| PTB     | 1.081       | +0.13       | 46.0            | 45.8            |
| WT2     | 1.081       | -0.03       | 40.9            | 42.3            |

**Table 7:** Cross-dataset summary for the LM transfer result.

| Metric                    | PTB    | WT2    |
|---------------------------|--------|--------|
| HOPF / BASELINE PPL ratio | 1.081× | 1.081× |
| HOPF - PERMUTED PPL       | +0.13  | -0.03  |
| HOPF effective-path ratio | 1.39×  | 1.56×  |
| BASELINE eff. paths       | 43.2   | 35.4   |
| HOPF eff. paths           | 46.0   | 40.9   |
| PERMUTED eff. paths       | 45.8   | 42.3   |

**Table 8:** Auxiliary-loss control supporting the main baseline choice. The learned sparse model with explicit balance loss is substantially less concentrated than the no-aux baseline.

| Condition      | Val. PPL | Eff. paths | Params |
|----------------|----------|------------|--------|
| BASELINE       | 154.55   | 44.0       | 5.66M  |
| LEARNED_SPARSE | 154.78   | 62.8       | 5.66M  |
| HOPF           | 165.59   | 47.7       | 5.64M  |
| PERMUTED       | 165.25   | 47.1       | 5.64M  |

**Table 9:** Full control and null-result summary from the canonical generated assets.

| Case                   | HOPF / BASE | HOPF<br>PERM | - Aux eff. paths | Interpretation                                                                                                     |
|------------------------|-------------|--------------|------------------|--------------------------------------------------------------------------------------------------------------------|
| PTB                    | 1.081       | +0.13        | –                | Hopf and permuted are effectively indistinguishable on PTB quality.                                                |
| WT2                    | 1.081       | -0.03        | –                | The same null result replicates on WT2.                                                                            |
| PTB_aux_control_screen | 1.071       | +0.34        | 62.8             | Load-balancing aux loss drives routing toward near-uniform usage, so no-aux top-1 is the correct paper comparator. |

**Table 10:** PTB detail table exported from canonical analysis results.

| Condition | Val. PPL | Eff. paths | $64/\text{eff}_b$ | Params |
|-----------|----------|------------|-------------------|--------|
| BASELINE  | 152.26   | 43.19      | $1.48\times$      | 5.66M  |
| HOPF      | 164.54   | 45.96      | $1.39\times$      | 5.64M  |
| PERMUTED  | 164.41   | 45.81      | $1.40\times$      | 5.64M  |

**Table 11:** WikiText-2 detail table exported from canonical analysis results.

| Condition | Val. PPL | Std. PPL | Eff. paths | $64/\text{eff}_b$ |
|-----------|----------|----------|------------|-------------------|
| BASELINE  | 105.85   | 1.57     | 35.37      | $1.81\times$      |
| HOPF      | 114.45   | 1.14     | 40.94      | $1.56\times$      |
| PERMUTED  | 114.48   | 0.87     | 42.28      | $1.51\times$      |